



I told a chat to remind me. Now my phone won't let me forget.

A task tracker built in one morning, with no paid apps, driven entirely from a conversation.

The wish list



Capture from anywhere

Phone or laptop, mid-thought: “remind me to pay the race entry Tuesday at 2.” No forms, no app-switching.



Nag until it's actually done

Not one polite ping. Every 30 minutes, until I tick it off.



Done from the phone

Tick it in the notification and the whole system knows.



Zero subscriptions

No Todoist Pro, no new accounts. Things I already own.

What it feels like

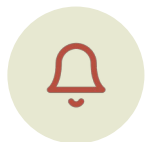
A real one from this morning.



11:42

I type

“book the dentist, set
reminder 11:55”



11:55

Phone buzzes

[T005] book the dentist



11:56

I tick it

Reminder disappears. Task
closed. Logged.

Had I ignored it, it would have come back at 12:25, 12:55, 13:25...

One morning, by the numbers

1

morning
(10:27 → 12:57)

0 €

spent

7

tasks captured

1

closed from
the phone

Things that were already lying around and got repurposed:

a laptop · Google Drive · Apple Reminders · Python · a chat window

The trick: three jobs, three tools

A chat can't run in the background — it only exists while you're typing. So it can't be the alarm clock. We gave each job to the thing that's good at it.



The brain

Claude, in chat

Understands “Tuesday at 2, pay Alex back” and turns it into a precise instruction.



The clock

A tiny program on the laptop

Wakes every minute, reads new instructions, decides who needs nagging.



The messenger

Apple Reminders

Already on the phone, already syncs, already has a tick-box. Every open task is pre-armed there, so the first alert fires even if the laptop sleeps.

You just say it

No forms, no fields, no app to open. Plain sentences, in whatever language you think in.

“remind me to buy toothpaste at 5”

› task · today 17:00 · nags every 30 min

“pay the race entry tomorrow 2pm”

› task · tomorrow 14:00

“Sam is starting the QC report”

› task for Sam · in progress · silent for you

“idea: rotate morning tasks”

› captured · no time · never nags

“move toothpaste to 3pm”

› due changed · phone reminder moves with it

“done toothpaste” · or just tick it on your phone

› closed · reminder gone · logged

“what’s open?” · “what’s Sam on?”

› live answer from the daemon’s status

If it can't tell which task you mean, it asks. Otherwise it acts and shows one line you can correct.

What you'll actually see



On your phone

A list called Claude in Reminders. Each task is there with its alert already set — it buzzes at the due time even if the laptop is asleep.



Every 30 minutes

The same item re-alerts. No duplicates, no pile-up. Quiet hours 22:00–07:00: nothing buzzes.



When you tick it

Gone from your phone within a minute. Marked done with a timestamp. Nobody has to tell anybody.

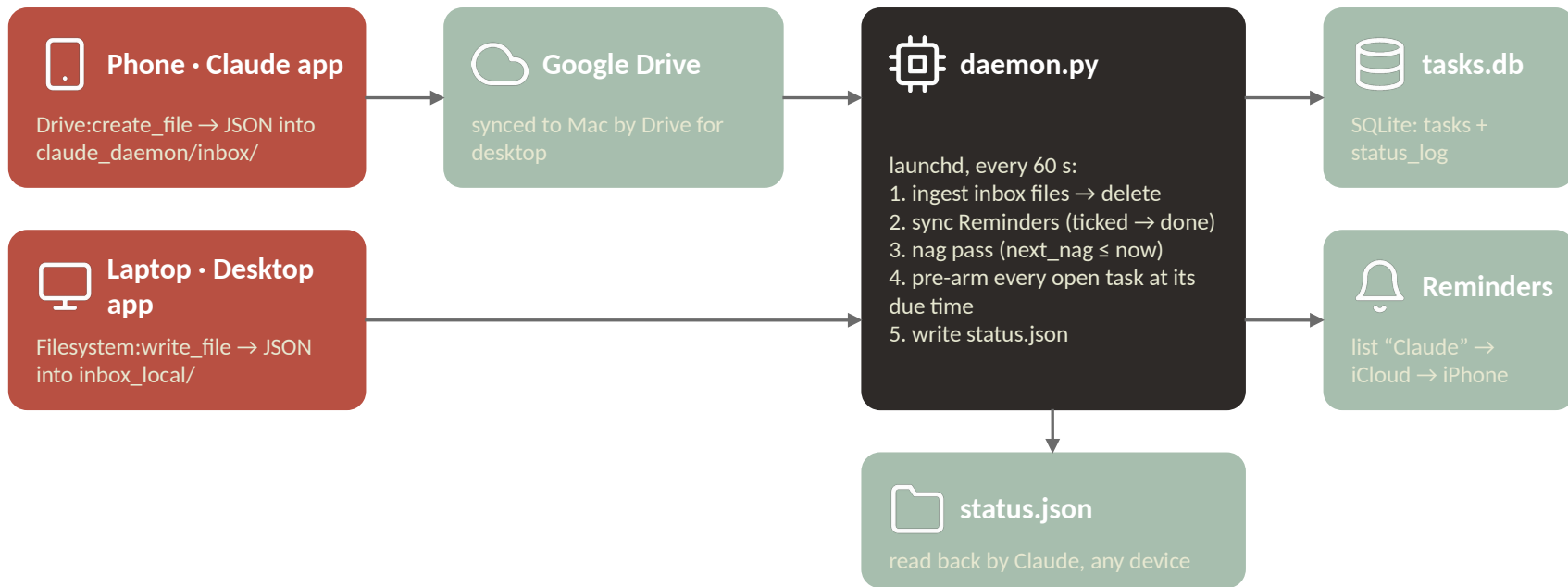


In the background

Everything — who, what, when, how long — sits in one small database. Export a month for a client as CSV in one sentence.

Three rules: only your tasks nag you • no time means no nagging • tick it or tell it, either works.

How it actually works



One rule makes it possible: Claude never talks to a running process, it only creates files. The daemon reads files and writes files. That is the entire contract — and why it can move to a Raspberry Pi unchanged.

The mailbox protocol

Every instruction is one small JSON file. Filename carries a timestamp so they're processed in order. Consumed, then deleted — the mailbox is empty in steady state.

```
{ "action": "add", "title": "pay the race entry",  
  "due": "2026-09-01T14:00" }  
  
{ "action": "add", "title": "batch effect QC report",  
  "assignee": "Sam", "status": "in_progress" }  
  
{ "action": "update", "id": "T001", "due": "2026-08-31T15:00" }  
  
{ "action": "done", "id": "toothpaste" }      ← fuzzy title ok  
  
{ "action": "snooze", "id": "T003", "minutes": 90 }  
  
{ "action": "export", "filter": { "project": "work" } } → CSV
```

Why create-only?

Phone-Claude can create Drive files but never edit them. One file per command sidesteps that entirely.

Why delete?

Nothing accumulates in Drive. The database is the record; the mailbox is transit.

Why fuzzy ids?

“done toothpaste” is how people talk. Ambiguous → the daemon refuses and lists candidates.

Status back

status.json is rewritten every tick and mirrored to Drive, so the phone can ask “what's open?”

The nag loop and the data behind it



Pre-arm at creation

Every open task of mine gets a Reminders item alerting at its due time, the moment it exists. The phone shows the whole list; the first alert needs no Mac.



$\text{next_nag} \leq \text{now?} \rightarrow \text{nag}$

macOS notification, then the same Reminders item is re-armed to $\text{now}+60$ s. Only my tasks with a due date; other people's never nag me.



$\text{next_nag} += 30 \text{ min}$

Skips 22:00–07:00 (quiet hours). Snooze or a new due date just moves next_nag — and the Reminder follows.



Tick anywhere $\rightarrow$ done

Reverse sync each tick: completed items become $\text{status}=\text{done}$ and are deleted from Reminders. Done via chat deletes them too.

tasks.db (SQLite)

tasks

```
id · title · kind · project  
assignee · status · due  
nag_every_min · next_nag  
reminder_id ·  
created/updated/done_at
```

status_log

```
task_id · from → to · changed_at
```

Every status change is a row. That is what makes “who worked on what, from when to when” a query — and a client report a CSV export.

What's next



Phone round-trip test

Add a task from the phone, watch it land on the laptop within a minute.



A dashboard

Timeline of who's on what, open items, tick, add — one HTML page served by the daemon.



A Raspberry Pi

So the clock never sleeps. Same code, same files, one box that's always on.



Team view

Same database, more names in the assignee column. Workload per person as a CSV for clients.

The AI didn't run anything.

It wrote small files, and a small program did the rest.

That's the whole design — and it's why the next version can live on a 60 € computer the size of a credit card.

Questions?